

高级语言程序设计

实验报告

南开大学计算机大类

姓名 王梓杰

学号 2514247

班级 软件二班

完成日期 2026 年 5 月 10 日

目录

1	作业题目	3
2	开发软件	3
3	课题要求	3
4	主要流程	4
4.1	整体流程与架构设计	4
4.2	核心算法	4
4.3	AI 工具披露	5
5	单元测试	6
6	收获与总结	6

1、作业题目

本次大作业的题目为：基于 Qt 的下落式音乐节奏游戏。要求实现一个具有图形界面的音游程序，音游谱面可以由玩家手动编辑或自动生成，并根据下落的音符在判定线处按键击打，系统根据击打的准确度进行计分和连击统计。

2、开发软件

Qt 6.11

3、课题要求

本课题需要实现以下核心功能：

1. 美观的无边框窗口：自定义标题栏，包含最小化和关闭按钮，支持窗口拖动；整体采用深色钢琴主题风格。
2. 音乐导入功能：支持导入 mp3、wav、flac、ogg 等常见音频格式，使用 Qt Multimedia 模块进行解码和播放。
3. 手动谱面编辑：提供可视化编辑器，支持通过鼠标点击或数字键快捷键在 5 条轨道上放置音符，可设置音符颜色（蓝色/黄色）和持续时间（短按/长按）。
4. 下落式游戏玩法：游戏界面包含 5 条轨道（对应键盘 A、D、G、J、L），白色判定线位于底部，音符实体从上方下落。
5. 判定系统：蓝色音符按对应轨道键击打，黄色音符统一按空格键击打；短按音符按键即消失，长按音符需持续按住直至尾部通过判定线；误差窗口为 $\pm 80\text{ms}$ 。
6. 计分系统：每个音符基础分 10 分，成功击中累加得分并增加连击数，未击中则连击清零并计为 Miss。
7. 谱面持久化：谱面以 JSON 格式保存，包含关联的音乐文件路径和所有音符的轨道、颜色、时间和持续时间信息。

4、主要流程

4.1 整体流程与架构设计

本项目采用经典的三层架构，将数据、视图和控制逻辑分离，确保代码结构清晰、易于维护和扩展。

(1) 数据层：由 Note 结构体和 Chart 类组成。Note 定义了单个音符的全部属性（轨道、颜色、时间、持续时间、击中状态）；Chart 是音符的集合容器，负责谱面的 JSON 序列化与反序列化。

(2) 视图层：由三个核心界面组成。Widget 为主控窗口，提供无边框外壳和界面路由；GameWidget 负责游戏时的实时渲染，包括钢琴背景、轨道、判定线、下落音符、击中特效和计分 UI；EditorWidget 提供可视化制谱界面，以时间轴方式展示和编辑音符。

(3) 控制层：通过 Qt 的信号与槽机制实现界面间的松耦合通信。Widget 作为中央控制器，接收用户操作并调度 GameWidget 和 EditorWidget 的状态切换；GameWidget 内置 60fps 定时器，每 16ms 刷新一次画面并执行判定逻辑。

4.2 核心算法

以下介绍核心数据结构、JSON 解析、渲染计算与判定逻辑。

(1) Note 数据结构

Note 结构体包含 track (0~4 轨道)、color (Blue/Yellow 枚举)、startTimeMs (触发时间)、durationMs (0 为短按， ≥ 200 为长按)、hit 和 missed (判定状态)。

Chart 类负责谱面的存储与读取。saveToFile 将 notes 数组转为 QJsonArray，每个 Note 写入 track、color 字符串、startTime 和 duration 字段。loadFromFile 反向解析。

(2) JSON 序列化与反序列化

使用 toVariant().toLongLong() 避免大整数溢出，加载后重置 hit/missed 为 false。

(3) 音符下落位置计算

音符 Y 坐标由时间差乘以像素速度计算：

$$\text{noteY} = \text{judgeLineY} - (\text{note.startTimeMs} - \text{now}) \times \text{PIXELS_PER_MS}$$

其中 $\text{PIXELS_PER_MS} = 0.45$, judgeLineY 固定为 $\text{height} - 120$ (判定线位置)。

(4) 判定窗口算法

判定窗口为 $\pm 80\text{ms}$ (JUDGE_WINDOW_MS)。计算 $\text{diff} = \text{note.startTimeMs} - \text{currentTime}$, 若 $|\text{diff}| \leq 80$ 且按键正确则命中；若 $\text{diff} < -80$ 则标记 missed。

长按音符需额外检测尾部： $\text{endTime} = \text{startTimeMs} + \text{durationMs}$, 尾判窗口同样为 $\pm 80\text{ms}$ 。

(5) 短按与长按判定逻辑

短按用 `m_keys.JustPressed` 检测本帧新按下的键，防止按住不放连续触发。长按分头判（在窗口内按下记录 `holdingNoteIndex`）、按住中（持续检测键状态）、尾判（结束窗口内仍按住则命中）三个阶段。

`m_holdingNoteIndex[5]` 数组记录各轨道当前按住的长音符索引，-1 表示无长按。判定结束后清空 `m_keys.JustPressed`。

4.3 AI 工具披露

在本项目的开发和报告撰写过程中，使用了以下 AI 工具辅助完成部分工作：

AI 工具	版本	具体用途	使用位置
Kimi (Moonshot AI)	Kimi K2.5	代码开发与架构设计	widget.cpp/h、editorwidget.cpp/h、chart.cpp/h、note.h
Kimi (Moonshot AI)	Kimi K2.5	算法逻辑推导	长短按逻辑

说明：以上 AI 工具主要用于辅助代码框架搭建、算法逻辑梳理所有核心算法逻辑均经过人工审核与调整，代码功能在提交前进行了完整的功能验证。

5、单元测试

针对游戏的核心模块设计了以下单元测试案例，覆盖音符判定、谱面存取和游戏流程：

序号	测试内容	预期结果	结果
1	导入 mp3 音乐文件并播放	音乐正常播放，进度条随时间前进	通过
2	在编辑器中点击轨道放置蓝色短音符	音符出现在正确轨道和时间位置	通过
3	短音符在 $\pm 80\text{ms}$ 内按键击打	note.hit=true, Score+10, Combo+1	通过
4	短音符在 $\pm 80\text{ms}$ 外按键或 miss	note.missed=true, Combo 清零	通过
5	长按音符在判定窗口内按下并持续按住	尾部通过后 note.hit=true	通过
6	长按音符提前松开	note.missed=true, Combo 清零	通过
7	黄色音符统一按空格键击打	所有轨道的黄色音符均响应 Space 键	通过
8	游戏结束后返回菜单	音乐停止，界面切回主菜单	通过

测试结论：所有 8 项测试均已通过，游戏核心功能运行正常。在开发过程中曾遇到长按音符判定逻辑不完整的问题（松开按键后未正确标记 Miss），通过引入 m_holdingNoteIndex 状态数组和按键释放监听，成功修复了该问题。

6、收获与总结

通过本次大作业的完整开发过程，我在以下几个方面获得了显著提升：

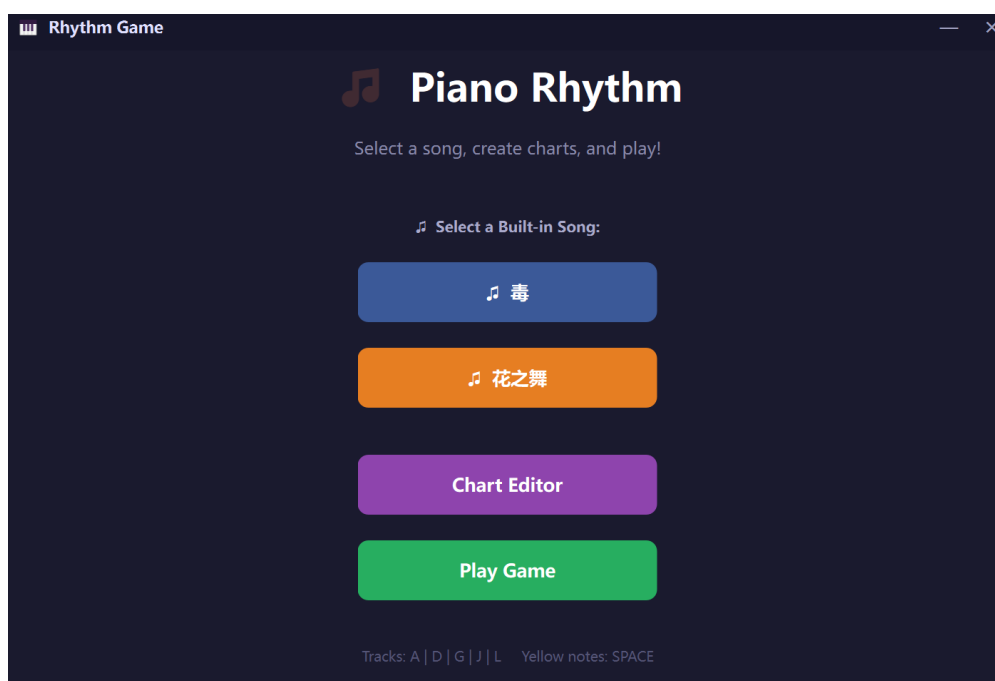
1. Qt 框架的深入理解：掌握了 QMediaPlayer 与 QAudioOutput 的音频播放机制、QPainter 的自定义绘制流程、QStackedWidget 的多界面路由管理，以及 Qt 信号与槽的松耦合通信模式。
2. 状态机与防抖动设计：长按音符的判定本质上是一个小型状态机（未开始 $\rightarrow$ 按住中 $\rightarrow$ 成功/失败）。同时，通过区分 m_keysPressed 和 m_keysJustPressed，解决了键盘自动重复导致的连击误触发问题。

3. 工程化开发规范：学习了 CMake 的现代项目配置方法、JSON 数据持久化格式设计、以及代码分层架构（数据层-视图层-控制层）在实际项目中的应用。

总体而言，本次项目将高级语言程序设计课程中的面向对象、多态、文件 I/O、数据结构等知识点，与 Qt 图形编程、实时系统设计和人机交互等工程实践紧密结合，是一次非常完整且有价值的综合训练。

附录

游戏界面截图：



Github 仓库：<https://kkgithub.com/mathlover0728/music-game.git>